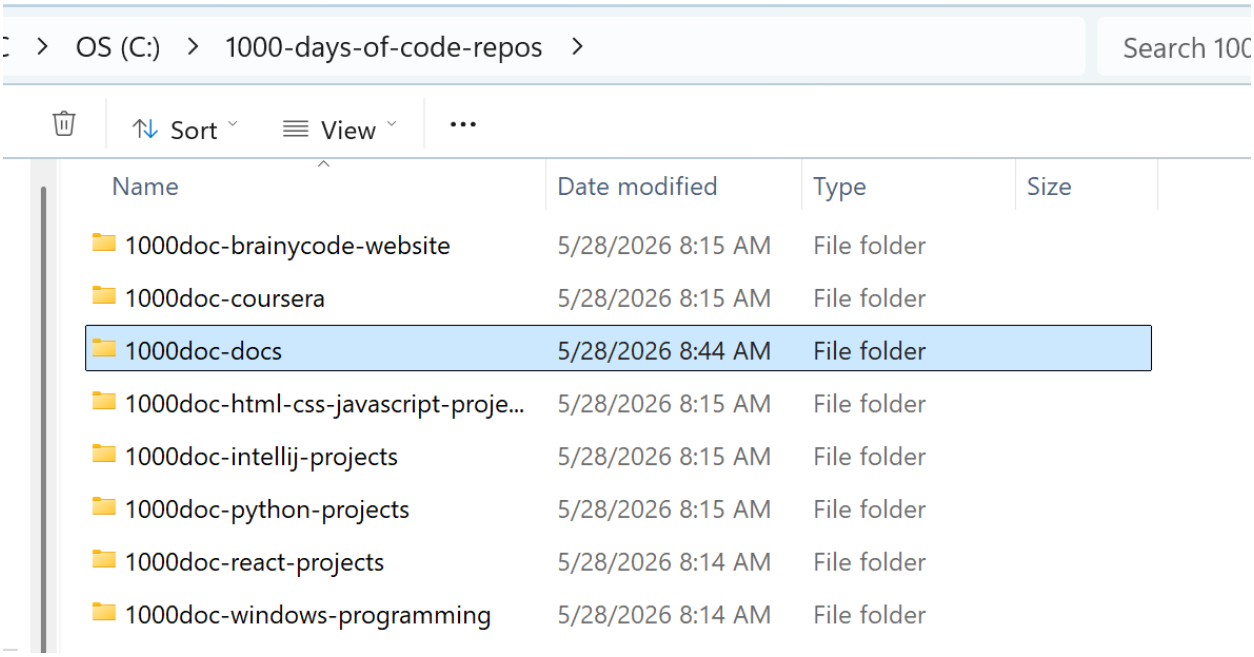
Update All Repos Script

Purpose

In the past, I only ran one bat file: update_git.bat to update the one big repo I maintained for 1000-days-of-code. Having split the huge repo into several:



The goal is to have all related work in the same repo rather than one huge repo that was taking longer and longer to update. But, the problem now is that I need to update/improve from one simple script:

```
@echo off
REM Move to target directory
cd C:\1000-days-of-code
REM Print out status first
git status
REM Wait to continue
pause
REM Add all modified and new files to git
git add .
REM Get commit message
set /p "msg=Commit Message: "
REM Commit
git commit -m %msg%
```

```
REM upload to repo
git push
```

Into one that checks each repo (even the original one) and if there are any changes updates it to github.

This new script will replace update_git.bat above.

What new script should do

For each repo folder:

1. cd into the repo
2. Run git status --porcelain
3. If output is empty → **no changes**, skip
4. If there *are* changes:
    - show the status
    - ask for a commit message
    - run git add .
    - run git commit -m "<msg>"
    - run git push
5. Move on to the next repo

The Script

```
import os
import subprocess

# List of repo directories (absolute paths recommended)
REPOS = [
    r"C:\1000-days-of-code",    # original repo
    r"C:\1000-days-of-code-repos\1000doc-brainycode-website",
    r"C:\1000-days-of-code-repos\1000doc-coursera",
    r"C:\1000-days-of-code-repos\1000doc-docs",
    r"C:\1000-days-of-code-repos\1000doc-html-css-javascript-projects",
    r"C:\1000-days-of-code-repos\1000doc-intellij-projects",
    r"C:\1000-days-of-code-repos\1000doc-python-projects",
    r"C:\1000-days-of-code-repos\1000doc-react-projects",
    r"C:\1000-days-of-code-repos\1000doc-windows-programming",
]
```

```
def run(cmd, cwd):
    """Run a shell command inside a specific directory and return output."""
    result = subprocess.run(cmd, cwd=cwd, capture_output=True, text=True, shell=True)
    return result.stdout.strip()


def update_repo(path):
    print(f"\n=== Checking repo: {path} ===")

    # Check for changes
    status = run("git status --porcelain", path)

    if not status:
        print("No changes. Skipping.")
        return

    print("Changes detected:")
    print(run("git status", path))

    msg = input("Commit message (leave empty to skip): ").strip()
    if not msg:
        print("Skipping commit for this repo.")
        return

    # Commit and push
    run("git add .", path)
    run(f'git commit -m "{msg}"', path)
    print(run("git push", path))

    print("Updated successfully.")


def main():
    for repo in REPOS:
        if os.path.isdir(repo):
            update_repo(repo)
        else:
            print(f"Directory not found: {repo}")


if __name__ == "__main__":
    main()
```

How the Script Works

✓ Only commits repos with changes

It uses:

```
1. git status --porcelain
```

This returns **empty output** if nothing changed.

✓ Asks for a commit message only when needed

If you press Enter with no message, it skips that repo.

✓ **Works for any number of repos**

Just add paths to the REPOS list.

✓ **Safe — won't break anything**

It only commits if you explicitly give a message.

How to run it

1. Save the script as:

```
update_all_repos.py
```

2. Run it from anywhere

```
python update_all_repos.py
```

3. Create a .bat wrapper:

```
python C:\path\to\update_all_repos.py
```

Some Notes

QUESTION: Why are the folder names prefixed with `r` as in `r"C:\1000-days-of-code"`?

The `r` makes the string a **raw string literal**.

A *raw string* tells Python:

"Interpret backslashes **literally**, not as escape characters."

This matters because Windows paths contain **lots of backslashes**, and Python normally treats backslashes as escape sequences.

⭐ **When should you use raw strings for paths?**

Use r"..." when:

- the path contains backslashes
- you want to avoid escaping them
- you're writing Windows paths in Python scripts

You *don't* need raw strings for:

- Linux/macOS paths (/home/...)
- Windows paths using forward slashes (C:/1000-days-of-code)
- Paths built with os.path.join()

⭐ Your script will work fine with or without r if you use forward slashes

Python accepts:

```
1. "C:/1000-days-of-code"
```

This is perfectly valid on Windows.

But the raw string is still the cleanest and safest.

QUESTION: More information on how subprocess.run works

Let's walk through **each argument** in

```
1. subprocess.run(cmd, cwd=cwd, capture_output=True, text=True, shell=True)
```

⭐ subprocess.run() — What each argument does

1. cmd

This is the command you want to run, e.g.:

```
1. "git status"
2. "git add ."
3. "git commit -m 'message'"
```

It's passed as a string because we set shell=True.

2. cwd=cwd

This tells Python:

"Run the command **inside this directory**."

So if cwd is:

```
1. C:\python-projects
```

then the command runs *as if you opened a terminal in that folder*.

This is how the script jumps between repos.

3. capture_output=True

This tells Python:

“Capture the command’s stdout and stderr instead of printing them.”

So instead of the output going to the console, you get it back as a string:

```
result.stdout
result.stderr
```

This is how the script checks whether git status has changes.

4. text=True

This tells Python:

“Return output as **text strings**, not bytes.”

Without this, you’d get byte strings like:

```
1. b'On branch main\nnothing to commit\n'
```

With text=True, you get clean strings:

```
1. "On branch main\nnothing to commit\n"
```

Much easier to work with.

5. shell=True

This tells Python:

“Run the command through the system shell.”

On Windows, that means:

- cmd.exe

- or PowerShell, depending on settings

This allows commands like:

```
git status
git add .
git commit -m "msg"
```

to run exactly as they would in a normal terminal.

Without shell=True, you'd have to pass commands as lists:

```
1. ["git", "status"]
```

Using shell=True keeps the script simple and readable.

⭐ Putting it all together

This line:

```
1. result = subprocess.run(cmd, cwd=cwd, capture_output=True, text=True, shell=True)
```

means:

- run the command in the given repo folder
- capture the output
- return it as text
- run it through the normal shell
- don't print anything automatically

And then:

```
1. return result.stdout.strip()
```

gives you the clean output.

⭐ Why this design works beautifully for your multi-repo updater

It lets you:

- hop into each repo

- run git commands
- check for changes
- commit only when needed
- push automatically

All without ever leaving Python.

Issues

Issue #1: The script did not update GitHub.

The reason was found to be associated with the fact that I usually enter my commit message with a starting and ending quote, e.g. “This is a test”.

The problem is that this breaks the commit part because the code uses:

```
run(f'git commit -m "{msg}"', path)
```

The above get translated into git commit -m ““This is a test”” which generates an error when trying to commit and then fails to execute the push.

Solution:

Change to:

```
1. subprocess.run(["git", "commit", "-m", msg], cwd=path)
```

Issue #2: The repo name gets lost in all the output.

Add color:

```
CYAN = "\033[96m"
RESET = "\033[0m"
```

And

```
print(f"\n{CYAN}=== Checking repo: {path} ==={RESET}")
```

Did not work. I had to enter:

```
reg add HKCU\Console /v VirtualTerminalLevel /t REG_DWORD /d 1 /f
```

I closed the CMD prompt and opened a new one and got:

```
No changes. Skipping.

=== Checking repo: C:\1000-days-of-code-repos\1000doc-intellij-projects ===
No changes. Skipping.

=== Checking repo: C:\1000-days-of-code-repos\1000doc-python-projects ===
Changes detected:
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   update_all_repos/update_all_repos.py

no changes added to commit (use "git add" and/or "git commit -a")
Commit message (leave empty to skip): "Adding color so repo name stands out"
...Updating the repo: C:\1000-days-of-code-repos\1000doc-python-projects
[main 16f39bf] "Adding color so repo name stands out"
 1 file changed, 4 insertions(+), 1 deletion(-)

Updated successfully.

=== Checking repo: C:\1000-days-of-code-repos\1000doc-react-projects ===
No changes. Skipping.

=== Checking repo: C:\1000-days-of-code-repos\1000doc-windows-programming ===
No changes. Skipping.
```

Final Script Code

```python
import os
import subprocess

# List of repo directories (absolute paths recommended)
REPOS = [
    r"C:\1000-days-of-code",   # original repo
    r"C:\1000-days-of-code-repos\1000doc-brainycode-website",
    r"C:\1000-days-of-code-repos\1000doc-coursera",
    r"C:\1000-days-of-code-repos\1000doc-docs",
    r"C:\1000-days-of-code-repos\1000doc-html-css-javascript-projects",
    r"C:\1000-days-of-code-repos\1000doc-intellij-projects",
    r"C:\1000-days-of-code-repos\1000doc-python-projects",
    r"C:\1000-days-of-code-repos\1000doc-react-projects",
    r"C:\1000-days-of-code-repos\1000doc-windows-programming",
]

CYAN = "\033[96m"
RESET = "\033[0m"


def run(cmd, cwd):
    """Run a shell command inside a specific directory and return output."""
    result = subprocess.run(cmd, cwd=cwd, capture_output=True, text=True, shell=True)
    return result.stdout.strip()


def update_repo(path):
```

```
    print(f"\n{CYAN}=== Checking repo: {path} ==={RESET}")

    # Check for changes
    status = run("git status --porcelain", path)

    if not status:
        print("No changes. Skipping.")
        return

    print("Changes detected:")
    print(run("git status", path))

    msg = input("Commit message (leave empty to skip): ").strip()
    if not msg:
        print("Skipping commit for this repo.")
        return

    # Commit and push
    print(f"...Updating the repo: {path}")
    run("git add .", path)
    #run(f'git commit -m "{msg}"', path)
    subprocess.run(["git", "commit", "-m", msg], cwd=path)
    print(run("git push", path))

    print("Updated successfully.")


def main():
    for repo in REPOS:
        if os.path.isdir(repo):
            update_repo(repo)
        else:
            print(f"Directory not found: {repo}")


if __name__ == "__main__":
    main()
```